SOFTWARE TESTING & QUALITY ASSURANCE

Name: _____ **Stream:** **BSc. Computer Science (Sem-V)** **Roll No.:** ____

INDEX

Sr.	Practical Name	Date	Signature
1	Creation of Test Suite Using Selenium IDE		
2	Cross-Website Functional Testing Using Selenium IDE		
3	Manual Test Case Design and Execution		
4	Selenium Server (Grid) Configuration and Remote Execution		
5	Login Automation and Validation using Selenium WebDriver (Java)		
6	Web Element Interaction and Synchronization Handling		
7	Data-Driven Testing Using Excel Integration		
8	Object Identification and Counting on Web Page		
9	List and Combo Box Item Verification		
10	Checkbox Identification and Validation		
11	Dynamic Web Table Handling and Sorting Validation		
12	File Upload and Download Automation		
13	Screenshot Capture and Logging Mechanism		
14	Automation testing framework using Page Object Model (POM) with Selenium WebDriver and TestNG		
15	TestNG Framework Implementation and Reporting		

1 Practical No. 1: Creation of Test Suite Using Selenium IDE

Aim

To install Selenium IDE and create a test suite containing a minimum of four test cases to validate different web page formats (HTML page, XML response, JSON API output, and Dynamic Form page). Execute the test suite and export the automation script in WebDriver format.

Theory

- **Selenium IDE:** An open-source browser extension used for automating web application testing. It allows users to record, edit, and replay test cases without writing programming code.
- **Test Case:** A sequence of actions performed to verify a specific functionality of a web application.
- **Test Suite:** A collection of multiple test cases that are executed together.

Requirements

- Google Chrome or Mozilla Firefox
- Selenium IDE Extension
- Internet Connection

Websites Used

Test Case	Website
HTML Page	`https://example.com`
XML Response	`https://www.w3schools.com/xml`
JSON API	`https://jsonplaceholder.typicode.com`
Dynamic Form	`https://www.selenium.dev/selenium/web/web-form`

Procedure

Step 1: Install Selenium IDE

1. Open Firefox or Google Chrome.
2. Install Selenium IDE from the browser extension store.
3. Open Selenium IDE.
4. Click **Create New Project**.
5. Enter the Project Name: `Web Page Validation`.
6. Click **Start Recording**.

Test Case 1 – HTML Page Validation

- **Website:** `https://example.com`
- **Objective:** To verify that the HTML page opens correctly.
- **Steps:**
    1. Open Selenium IDE.
    2. Create a new test named `HTML_Test`.
    3. Open the website.
    4. Verify the page title.
    5. Click **More information...**
    6. Stop recording.
- **Expected Output:**
    - HTML page opens successfully.
    - Title matches “Example Domain”.
    - “More information...” link opens the next page.

Test Case 2 – XML Response Validation

- **Website:** `https://www.w3schools.com/xml`
- **Objective:** To verify the XML response.
- **Steps:**
    1. Create a new test.
    2. Open the XML URL.

3. Verify that the XML content is displayed.
4. Stop recording.

- **Expected Output:** The XML document is displayed correctly.

Test Case 3 – JSON API Validation

- **Website:** `https://jsonplaceholder.typicode.com`
- **Objective:** To verify the JSON response.
- **Steps:**
  1. Create a new test.
  2. Open the JSON URL.
  3. Verify the JSON output.
  4. Stop recording.
- **Expected Output:** JSON data is displayed successfully.

Test Case 4 – Dynamic Form Validation

- **Website:** `https://www.selenium.dev/selenium/web/web-form`
- **Objective:** To validate the dynamic web form.
- **Steps:**
  1. Create a new test.
  2. Open the web form.
  3. Enter `Kiran` in the Text Box.
  4. Enter `12345` in the Password field.
  5. Select `Two` from the dropdown.
  6. Click **Submit**.
  7. Verify that the confirmation page is displayed.

Creating the Test Suite

1. Click **Test Suites**.
2. Click **+ New Test Suite**.
3. Enter the suite name: `Web Validation Suite`.
4. Add the following test cases: `HTML_Test`, `XML_Test`, `JSON_Test`, `Dynamic_Form_Test`.
5. Save the Test Suite.

Execution & Export

1. Click **Run All Tests**.
2. Observe the execution log.
3. Click the ⋮ **(three dots)** → **Export**.
4. Choose the required language (Java/Python/C#) and save the WebDriver script.

Expected Result

All four test cases execute successfully without errors, and the script exports perfectly in WebDriver format.

2 Practical No. 2: Cross-Website Functional Testing Using Selenium IDE

Aim

To develop and execute a test suite for two different websites using Selenium IDE. Perform functional operations such as clicking links, filling forms, selecting dropdown options, and verifying displayed content. Analyze the test results and generate execution logs.

Theory

- **Cross-Website Functional Testing:** A software testing technique used to verify that common user operations (such as clicking hyperlinks, entering text inputs, selecting dropdown options, submitting forms, and validating displayed content) function accurately across different web domains or applications.
- **Selenium IDE:** A browser extension that records and replays user actions automatically without requiring programming code, making it highly effective for automating repetitive testing flows.

Requirements

- Mozilla Firefox or Google Chrome

- Selenium IDE Extension
- Internet Connection

Websites Used

Website	Purpose
`https://selenium.dev`	Dynamic Form Testing
`https://example.com`	HTML Page and Link Testing

Procedure

Step 1: Open Selenium IDE

1. Open Mozilla Firefox or Google Chrome.
2. Launch the Selenium IDE extension.
3. Click **Create New Project**.
4. Enter the Project Name: `Cross Website Functional Testing`
5. Click **Start Recording**.

Test Case 1 – Dynamic Form Testing

- **Website:** `https://selenium.dev`
- **Objective:** To verify the functionality of a dynamic web form.
- **Steps:**
  1. Open the Selenium demo web form URL.
  2. Enter `Name` in the Text Box.
  3. Enter `12345` in the Password field.
  4. Select `Two` from the Dropdown menu.
  5. Click **Submit**.
  6. Verify that the confirmation page displays the message: `Received!`
- **Expected Output:**
  - Form opens successfully.
  - Text inputs are entered correctly.
  - Dropdown value is selected.
  - Form submits without errors.
  - The confirmation text `Received!` is verified on the UI.

Test Case 2 – HTML Page Testing

- **Website:** `https://example.com`
- **Objective:** To verify webpage title and hyperlink functionality.
- **Steps:**
  1. Open the Example Domain website.
  2. Verify that the page title matches expectations.
  3. Click the **More information...** link.
  4. Verify that the destination webpage opens successfully.
- **Expected Output:**
  - Example Domain page opens cleanly.
  - Webpage title is verified successfully.
  - “More information...” hyperlink redirects to the correct subsequent page.

Creating the Test Suite

1. Click the **Test Suites** tab in Selenium IDE.
2. Click the **+ New Test Suite** button.
3. Enter the Test Suite Name: `Cross Website Test Suite`
4. Add the following recorded test cases to it:
   - `Dynamic_Form_Test`
   - `HTML_Page_Test`
5. Save the Test Suite.

Executing the Test Suite

1. Click the **Run All Tests** icon.
2. Selenium IDE automatically executes both test cases in sequence.
3. Observe the live execution status checklist.
4. Review the final execution logs for debugging.

Expected Result

Both external target websites are tested successfully. All functional interface operations execute in sequence, proper execution logs are generated within the console, and all test cases return a passing status.

3 Practical No. 3: Manual Test Case Design and Execution

Aim

To prepare a Test Plan, Test Scenarios, and Test Cases for a web application, execute the test cases manually, and prepare a Test Execution Report and Defect Report according to the Software Testing Life Cycle (STLC).

Theory

- **Manual Testing:** The process of executing predefined test cases manually without using any automation tools to check if the application matches expected requirements. It helps identify functional defects, validation errors, missing requirements, and usability bugs.
- **Software Testing Life Cycle (STLC):** The systematic process followed to test software applications:
  1. Requirement Analysis
  2. Test Planning
  3. Test Case Design
  4. Test Environment Setup
  5. Test Execution
  6. Defect Reporting
  7. Test Closure

Requirements

- Microsoft Excel or Google Sheets
- Web Browser (Google Chrome)
- Internet Connection
- **Target Website:** `https://saucedemo.com`

Test Plan

Field	Details
Project Name	SauceDemo Login Testing
Application URL	`https://saucedemo.com`
Objective	To verify the Login functionality of the SauceDemo website
Testing Type	Manual Testing
Tester	Student Name
Testing Tool	MS Excel / Google Sheets
Start / End Date	11/07/2026

Test Scenarios

Scenario ID	Test Scenario	Description
TS01	Verify Login Page opens successfully	Check whether the Login page opens correctly.
TS02	Verify login with valid credentials	Check whether the user can log in using valid username and password.
TS03	Verify login with invalid password	Check whether an error message is displayed for an invalid password.

TS04	Verify login with invalid username	Check whether an error message is displayed for an invalid username.
TS05	Verify login with empty username	Check whether the username validation message is displayed.
TS06	Verify login with empty password	Check whether the password validation message is displayed.
TS07	Verify login with both fields empty	Check whether validation messages are displayed when both fields are empty.

Test Cases

TC ID	Test Case	Test Steps	Test Data	Expected Result
TC01	Login page verification	1. Open SauceDemo website.	N/A	Login page should open successfully.
TC02	Login with valid credentials	1. Enter valid username. 2. Enter valid password. 3. Click Login.	**Username:** `standard_user` **Password:** `secret_sauce`	User should be redirected to the Products page.
TC03	Login with invalid password	1. Enter valid username. 2. Enter invalid password. 3. Click Login.	**Username:** `standard_user` **Password:** `wrong123`	Error message should be displayed.
TC04	Login with invalid username	1. Enter invalid username. 2. Enter valid password. 3. Click Login.	**Username:** `test123` **Password:** `secret_sauce`	Error message should be displayed.
TC05	Login with empty username	1. Leave username blank. 2. Enter valid password. 3. Click Login.	**Username:** Blank **Password:** `secret_sauce`	“Username is required” message should appear.
TC06	Login with empty password	1. Enter valid username. 2. Leave password blank. 3. Click Login.	**Username:** `standard_user` **Password:** Blank	“Password is required” message should appear.
TC07	Login with both fields empty	1. Leave both fields blank. 2. Click Login.	**Username:** Blank **Password:** Blank	Validation message should be displayed.

Test Execution Report

TC ID	Test Case	Expected Result	Actual Result	Status	Comment
TC01	Login page verification	Login page opens successfully	Login page opened successfully	**PASS**	-
TC02	Login with valid credentials	Products page displayed	Products page displayed	**PASS**	-
TC03	Login with invalid password	Error message displayed	Error message displayed	**PASS**	-
TC04	Login with invalid username	Error message displayed	Error message displayed	**PASS**	-
TC05	Login with empty username	Username validation displayed	Username validation displayed	**PASS**	-
TC06	Login with empty password	Password validation displayed	Password validation displayed	**PASS**	-
TC07	Login with both fields empty	Validation message displayed	Validation message displayed	**PASS**	-

Execution Summary Table

Total Test Cases	Passed	Failed	Blocked
7	7	0	0

Overall Status: Testing Completed Successfully

Defect Report

- **Defect ID:** `BUG001`
- **Module:** Login Module
- **Description:** Login button accepts empty username in some cases.
- **Steps to Reproduce:**
    1. Open the Login page.
    2. Leave the username blank.
    3. Enter a password.
    4. Click Login.
- **Expected Result:** Username validation message should be displayed immediately.
- **Actual Result:** Login button becomes clickable before the validation message is triggered.
- **Severity:** Medium (Functionality works incorrectly but application is usable)
- **Priority:** High (Must be fixed immediately)
- **Status:** Open

Severity Levels

Severity	Meaning
Critical	Application crashes or major functionality fails.
High	Important functionality is not working.
Medium	Functionality works incorrectly but application is usable.
Low	Minor UI or cosmetic issue.

Priority Levels

Priority	Meaning
High	Must be fixed immediately.
Medium	Fix in the next release.
Low	Fix when time permits.

Expected Result
The Test Plan, Test Scenarios, and manual Test Cases are drafted successfully. Manual execution is completed against the web page application, resulting in a compiled Test Execution Report and an open Defect Report logged in strict adherence to STLC guidelines.

4 Practical No. 4: Selenium Server (Grid) Configuration and Remote Execution

Aim
To install and configure Selenium Server (Grid) and execute the same test script on multiple browsers (Chrome and Firefox) using Remote WebDriver.

Theory
- **Selenium Grid:** A tool within the Selenium suite that allows automation scripts to run on different browsers, operating systems, and machines simultaneously. It uses a Hub-Node architecture (or a unified standalone mode) to achieve distributed cross-browser testing.
- **Remote WebDriver:** An implementation class used to connect the local automation test script to a remote browser runner environment hosted on a Selenium Grid server, rather than launching local driver binaries directly on the host machine.
- **Cross-Browser Testing:** A verification technique performed to check whether a web application behaves consistently and renders correctly across various web browsers such as Google Chrome and Mozilla Firefox.

Tools Required
- Selenium Server (Grid) Standalone JAR File (`selenium-server-4.44.0.jar`)
- Eclipse IDE / IntelliJ IDEA
- Java JDK (Version 8 or above)
- Google Chrome Browser & ChromeDriver
- Mozilla Firefox Browser & GeckoDriver

Procedure

Step 1: Download the Selenium Server JAR

Download the latest or required Selenium Server standalone executable package file (`selenium-server-4.44.0.jar`) and place it in a dedicated working folder.

Step 2: Start Selenium Grid Server

1. Open the **Command Prompt** (Windows) or Terminal (macOS/Linux).
2. Navigate to the directory containing the downloaded JAR file using the `cd` command.
3. Start the Selenium Grid in `standalone` mode by running:

```
java -jar selenium-server-4.44.0.jar standalone
```

4. Wait for the terminal initialization log stream messages to confirm that the server is successfully running and listening on port `4444` (`http://localhost:4444`).

Step 3: Setup Project in Eclipse IDE

1. Launch **Eclipse IDE**.
2. Create a new Java Project: Go to **File** → **New** → **Java Project** and name it `SeleniumGridDemo` .
3. Create a new source package package framework named `grid` .
4. Create a new Java class within the package and name it `GridTest.java` .
5. Add the downloaded `selenium-server-4.44.0.jar` to the project's Build Path:
   - Right-click on the project → **Build Path** → **Configure Build Path**.
   - Click the **Libraries** tab → select **Classpath** → click **Add External JARs...**
   - Select your downloaded Selenium Server JAR file and click **Apply and Close**.

Step 4: Automation Script Implementation

Write the following code inside the `GridTest.java` file:

```
package grid;

import java.net.URL;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import org.openqa.selenium.firefox.FirefoxOptions;
import org.openqa.selenium.remote.RemoteWebDriver;

public class GridTest {
    public static void main(String[] args) {
        try {
            // Configure Chrome Options
            ChromeOptions chromeOptions = new ChromeOptions();

            // Initialize Remote WebDriver for Google Chrome
            WebDriver driver1 = new RemoteWebDriver(
                new URL("http://localhost:4444"),
                chromeOptions
            );

            // Execute test actions on Chrome
            driver1.get("https://google.com");
            System.out.println("Chrome Title: " + driver1.getTitle());
            driver1.quit();

            // Configure Firefox Options
            FirefoxOptions firefoxOptions = new FirefoxOptions();

            // Initialize Remote WebDriver for Mozilla Firefox
            WebDriver driver2 = new RemoteWebDriver(
                new URL("http://localhost:4444"),
                firefoxOptions
            );

            // Execute test actions on Firefox
            driver2.get("https://google.com");
            System.out.println("Firefox Title: " + driver2.getTitle());
            driver2.quit();

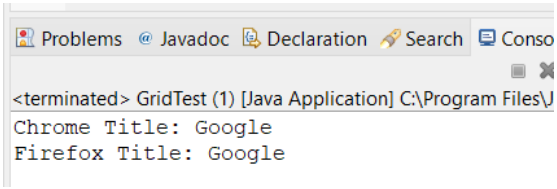
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Step 5: Run the Code

1. Save the program using `Ctrl + S` .
2. Right-click on `GridTest.java` in the project explorer tree pane.

3. Select **Run As** → **Java Application**.

Console Execution Output



Expected Result

The Selenium Server (Grid) engine initializes successfully on the loopback port `4444`. The compiled Java test automation program connects via `RemoteWebDriver` to distribute script commands, causing both Chrome and Firefox browsers to launch and pull down web content concurrently as expected.

5 Practical No. 5: Login Automation and Validation using Selenium WebDriver (Java)

Aim

To automate the login process of a web application using Selenium WebDriver and validate successful login using URL validation, welcome message validation, and logout button visibility.

Theory

- **Locators:** Selenium uses locators such as ID, Name, Class Name, and XPath to identify elements uniquely on a web page. In this practical, `By.id`, `By.tagName`, and `By.linkText` are utilized.
- **Validation Checkpoints:** Automated tests require verification checkpoints to confirm that an application is in the expected state. Validating the final redirected URL, capturing success header text elements, and confirming the presence of conditional elements (like a Logout link) ensures accurate test results.

Software Requirements

- Eclipse IDE / IntelliJ IDEA
- JDK 18 or above
- Google Chrome Browser & compatible ChromeDriver
- Selenium Server JAR (`selenium-server-4.44.0.jar`)

Target Website Details

- **URL:** `https://practicetestautomation.com`
- **Username:** `student`
- **Password:** `Password123`

Procedure

1. Open **Eclipse IDE**.
2. Create a new Java Project named `Login Automation`.
3. Configure the Project Build Path:
    - Right-click the project → **Build Path** → **Configure Build Path**.
    - Select **Libraries**, click **Add External JARs...**
    - Choose the `selenium-server-4.44.0.jar` file, then click **Apply and Close**.
4. Create a package named `login`.
5. Create a Java class named `LoginAuto` and ensure you select the checkbox for `public static void main(String[] args)`.
6. Write the automation code to fetch the page, fill the inputs, submit the form, and run validations.

Code Implementation

```java
package login;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;

public class LoginAuto {
```

```java
    public static void main(String[] args) {
        // Create WebDriver object
        WebDriver driver = new ChromeDriver();

        // Open website
        driver.get("https://practicetestautomation.com");

        // Maximize browser
        driver.manage().window().maximize();

        // Enter username
        driver.findElement(By.id("username")).sendKeys("student");

        // Enter password
        driver.findElement(By.id("password")).sendKeys("Password123");

        // Click Login button
        driver.findElement(By.id("submit")).click();

        // 1. URL Validation
        String expectedURL = "https://practicetestautomation.com";
        String actualURL = driver.getCurrentUrl();
        if (actualURL.equals(expectedURL)) {
            System.out.println("URL Validation Passed");
        } else {
            System.out.println("URL Validation Failed");
        }

        // 2. Welcome Message Validation
        WebElement message = driver.findElement(By.tagName("h1"));
        if (message.getText().equals("Logged In Successfully")) {
            System.out.println("Welcome Message Validation Passed");
        } else {
            System.out.println("Welcome Message Validation Failed");
        }

        // 3. Logout Button Validation
        WebElement logout = driver.findElement(By.linkText("Log out"));
        if (logout.isDisplayed()) {
            System.out.println("Logout Button is Visible");
        } else {
            System.out.println("Logout Button is Not Visible");
        }

        // Close browser
        driver.quit();
    }
}
```

Console Execution Output

Problems @ Javadoc Declaration Search Console ×

```
<terminated> LoginAuto [Java Application] C:\Program Files\Java\jdk-18\bin\javaw.exe  (22 Jul 2026, 12:31:07 am – 12:31:13 am) [pid: 17276]
Jul 22, 2026 12:31:10 AM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find CDP implementation matching 150
Jul 22, 2026 12:31:10 AM org.openqa.selenium.chromium.ChromiumDriver lambda$new$3
WARNING: Unable to find version of CDP to use for 150.0.7871.129. You may need to include a dependency
URL Validation Passed
Welcome Message Validation Passed
Logout Button is Visible
```

Expected Result

The automated script successfully opens the browser, interacts with the credentials text boxes, submits the login form, passes all three critical functional verification conditions, and terminates the driver session cleanly.

6 Practical No. 6: Web Element Interaction and Synchronization Handling

Aim

To automate interaction with various web elements such as textbox, password field, textarea, dropdown, checkbox, radio button, alert, and frame using Selenium WebDriver (Java) and implement implicit and explicit waits for synchronization handling.

Theory

- **Web Element Interaction:** Selenium provides various methods to interact with standard HTML elements. Common methods include `sendKeys()` for text inputs, `click()` for buttons, checkboxes, and radio buttons, and the `Select` class for handling dropdown menus.
- **Synchronization Handling:** Web applications often use asynchronous JavaScript or AJAX to load content dynamically. To prevent tests from failing due to timing issues, synchronization is handled using:

- **Implicit Wait:** Tells the WebDriver to poll the DOM for a certain duration (e.g., 10 seconds) when trying to find any element before throwing a `NoSuchElementException`.
- **Explicit Wait:** Configured using the `WebDriverWait` and `ExpectedConditions` classes to pause execution until a *specific conditional criteria* (e.g., visibility, element clickability) is fulfilled for a specific target locator.

Software Requirements

- Eclipse IDE / IntelliJ IDEA
- Java JDK 18 or above
- Google Chrome Browser & compatible ChromeDriver
- Selenium Server JAR (`selenium-server-4.44.0.jar`)

Websites Used

1. **Web Form:** `https://selenium.dev`
2. **Alert Demo:** `https://herokuapp.com`
3. **Frame Demo:** `https://herokuapp.com`

Procedure

1. Open **Eclipse IDE** and create a new Java Project named `WebElementAutomation`.
2. Add the Selenium Server JAR to your project configuration Build Path under Libraries.
3. Create a package named `webelement` and a class named `ElementInteraction` containing a standard `main` method execution loop.
4. Import the required Selenium library classes (`Alert` , `Select` , `WebDriverWait` , etc.).
5. Write the automation program to handle form submission, pop-up JavaScript alert handling, and frame switching sequences sequentially.

Code Implementation

```java
package webelement;

import java.time.Duration;
import org.openqa.selenium.Alert;
import org.openqa.selenium.By;
import org.openqa.selenium.Keys;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.Select;
import org.openqa.selenium.support.ui.WebDriverWait;

public class ElementInteraction {
    public static void main(String[] args) {
        // Launch Chrome Browser
        WebDriver driver = new ChromeDriver();

        // Configure Implicit Wait
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));

        // Maximize Browser
        driver.manage().window().maximize();

        // Configure Explicit Wait instance
        WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));

        // --- PART 1: Web Form Elements Interaction ---
        driver.get("https://selenium.dev");

        // Textbox
        driver.findElement(By.name("my-text")).sendKeys("Kiran Sahani");
        System.out.println("Textbox entered successfully");

        // Password
        driver.findElement(By.name("my-password")).sendKeys("12345");
        System.out.println("Password entered successfully");

        // Text Area
        driver.findElement(By.name("my-textarea")).sendKeys("Selenium Practical");
        System.out.println("Textarea entered successfully");

        // Dropdown Selection
        Select dropdown = new Select(driver.findElement(By.name("my-select")));
        dropdown.selectByVisibleText("Two");
        System.out.println("Dropdown selected successfully");

        // Checkbox
        driver.findElement(By.id("my-check-1")).click();
        System.out.println("Checkbox selected successfully");

        // Radio Button
        driver.findElement(By.id("my-radio-2")).click();
```

```
        System.out.println("Radio button selected successfully");

        // Submit Button Click
        driver.findElement(By.cssSelector("button")).click();
        System.out.println("Form submitted successfully");

        // --- PART 2: JavaScript Alert Handling ---
        driver.get("https://herokuapp.com");

        // Explicit wait applied to ensure clickability of alert button
        wait.until(ExpectedConditions.elementToBeClickable(
            By.xpath("//button[text()='Click for JS Alert']"))).click();

        // Switch to the Alert modal context overlay
        Alert alert = driver.switchTo().alert();
        alert.accept(); // Confirms and dismisses the alert pop-up
        System.out.println("Alert handled successfully");

        // --- PART 3: iFrame Handling ---
        driver.get("https://herokuapp.com");

        // Explicit wait to check frame availability and auto-switch into its context window
        wait.until(ExpectedConditions.frameToBeAvailableAndSwitchToIt(By.id("mce_0_ifr")));

        // Interact with elements located inside the swapped iframe window context
        WebElement frameText = wait.until(
            ExpectedConditions.visibilityOfElementLocated(By.id("tinymce")));

        frameText.sendKeys(Keys.CONTROL + "a");
        frameText.sendKeys(Keys.DELETE);
        frameText.sendKeys("Frame handled successfully.");

        // Switch back out to the parent main document page level context mapping
        driver.switchTo().defaultContent();
        System.out.println("Frame handled successfully");
        System.out.println("Practical executed successfully");

        // Close Browser Session
        driver.quit();
    }
}
```

Console Execution Output

```
Problems  Javadoc  Declaration  Search  Console ×
<terminated> ElementInteraction [Java Application] C:\Program Files\Java\jdk-18\bin\javaw.exe  (22 Jul 2026, 1:03:31 am – 1:03:40 am) [pid: 13572]
Jul 22, 2026 1:03:33 AM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find CDP implementation matching 150
Jul 22, 2026 1:03:33 AM org.openqa.selenium.chromium.ChromiumDriver lambda$new$3
WARNING: Unable to find version of CDP to use for 150.0.7871.129. You may need to include a dependency on a s
Textbox entered successfully
Password entered successfully
Textarea entered successfully
Dropdown selected successfully
Checkbox selected successfully
Radio button selected successfully
Form submitted successfully
Alert handled successfully
Frame handled successfully
Practical executed successfully
```

Expected Result

The script runs sequentially, seamlessly switching browser contexts between regular components, active JavaScript alert dialog models, and deep nested text iframes with synchronization intervals fully handled by the implicit and explicit configurations.

7 Practical No. 7: Data-Driven Testing Using Excel Integration

Aim

To update 10 student records in an Excel file and validate the data using Selenium WebDriver and Apache POI library.

Theory

- **Data-Driven Testing:** A software testing design pattern where the test data is separated from the source test script logic. Instead of hardcoding datasets inside the code, input configurations are read dynamically from external data repositories like spreadsheets, flat text files, or database tables.
- **Apache POI:** A powerful Java API library designed to create, read, modify, and update Microsoft Office documents programmatically. In automation architecture, it is predominantly used to read inputs from or output runtime results to Excel binary file wrappers (`.xls` and `.xlsx`).

Tools Required

- Java JDK (Version 8 or above)
- Eclipse IDE / IntelliJ IDEA
- Selenium WebDriver Server Libraries
- Google Chrome Browser & compatible ChromeDriver
- Apache POI Library (Version 5.2.3)
- Microsoft Excel (`.xlsx`) Spreadsheet file

Procedure

1. Open **Eclipse IDE**.
2. Create a new Java Project: **File** → **New** → **Java Project** and name it `StudentDataDriven`.
3. Create a clean package path named `studentdata`.
4. Create a Java class named `DataDrivenTest.java` containing a standard static main loop block.
5. Download the Apache POI external binary library distribution package (`Version 5.2.3`) from the archive:
   `https://apache.org`
6. Extract the downloaded ZIP archive file contents onto your local system space.
7. Import all core and dependent Apache POI external `.jar` binary components into your Eclipse Project Classpath dependencies:
   - Right-click the project folder → **Build Path** → **Configure Build Path**.
   - Click **Libraries** → **Classpath** → **Add External JARs...**
   - Multi-select all core POI, OOXML, and library dependencies, and hit **Apply and Close**.
8. Create a blank Excel ledger sheet named `StudentData.xlsx` in your target filesystem folder:
   `C:\Users\Kiran\OneDrive\Desktop\PVG\TYCS (STQA)\Practical\`
9. Implement the data streaming code, save, and run the automation routine.

Code Implementation

```
package studentdata;

import java.io.FileInputStream;
import java.io.FileOutputStream;
import org.apache.poi.ss.usermodel.*;
import org.apache.poi.xssf.usermodel.XSSFWorkbook;

public class DataDrivenTest {
    public static void main(String[] args) {
        try {
            // Define the absolute system path location of the Excel file
            String filePath = "C:\\Users\\Kiran\\OneDrive\\Desktop\\PVG\\TYCS (STQA)\\Practical\\StudentData.xlsx";

            // Open an input communication pipeline stream to connect to the file resource
            FileInputStream fis = new FileInputStream(filePath);
            Workbook workbook = new XSSFWorkbook(fis);
            Sheet sheet = workbook.getSheetAt(0);

            // Check if the first descriptive row layout header exists, create if not present
            Row header = sheet.getRow(0);
            if (header == null) {
                header = sheet.createRow(0);
                header.createCell(0).setCellValue("Roll Number");
                header.createCell(1).setCellValue("Name");
                header.createCell(2).setCellValue("Course");
                header.createCell(3).setCellValue("Mobile");
                header.createCell(4).setCellValue("Status");
            }

            // Define the data matrix array containing 10 student database entries
            String[][] students = {
                {"101", "Rahul Sharma", "B.Sc CS", "9876543210", "Pass"},
                {"102", "Priya Patel", "B.Sc IT", "9876543211", "Pass"},
                {"103", "Amit Verma", "BCA", "9876543212", "Fail"},
                {"104", "Sneha Singh", "B.Sc CS", "9876543213", "Pass"},
                {"105", "Rohan Mehta", "B.Sc IT", "9876543214", "Fail"},
                {"106", "Neha Gupta", "BCA", "9876543215", "Pass"},
                {"107", "Arjun Nair", "B.Sc CS", "9876543216", "Pass"},
                {"108", "Pooja Yadav", "B.Sc IT", "9876543217", "Fail"},
                {"109", "Vikas Kumar", "BCA", "9876543218", "Pass"},
                {"110", "Anjali Desai", "B.Sc CS", "9876543219", "Pass"}
            };

            // Loop through the data matrix array and map values to sequential spreadsheet rows
            for (int i = 0; i < students.length; i++) {
                // target the immediate trailing open row cell indexes (offset past header row)
                Row row = sheet.getRow(i + 1);
                if (row == null) {
                    row = sheet.createRow(i + 1);
                }

                // Write column properties values into cells programmatically
                row.createCell(0).setCellValue(students[i][0]);
                row.createCell(1).setCellValue(students[i][1]);
                row.createCell(2).setCellValue(students[i][2]);
```

```
                row.createCell(3).setCellValue(students[i][3]);
                row.createCell(4).setCellValue(students[i][4]);
            }

            // Close the input file parsing layer stream safely
            fis.close();

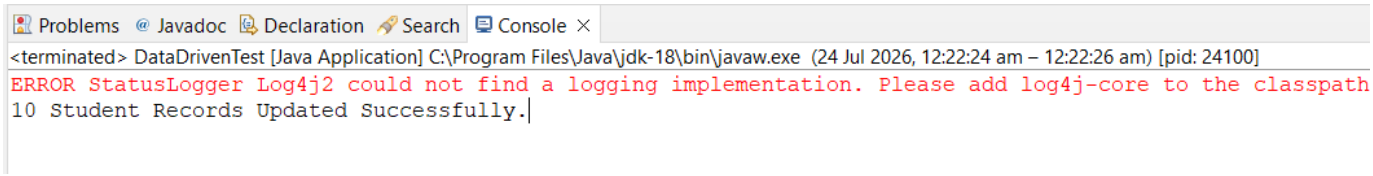
            // Open an output communication stream to rewrite data states to the physical disk file
            FileOutputStream fos = new FileOutputStream(filePath);
            workbook.write(fos);

            // Flushes data allocations and releases underlying document lock structures
            fos.close();
            workbook.close();

            System.out.println("10 Student Records Updated Successfully.");

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Console Execution Output



Expected Result

The automated Java framework successfully initiates connection paths to the spreadsheet workbook package via Apache POI libraries, maps structural metrics into cells sequentially, and saves all 10 updated data records seamlessly to the filesystem.

8 Practical No. 8: Object Identification and Counting on Web Page

Aim

To write a Java program using Selenium WebDriver to identify and count the total number of objects present on a web page, including links, images, buttons, and input elements, and display their categorized counts.

Theory

- **Object Identification:** The process of locating UI components on a web application screen. Selenium WebDriver provides the `findElements()` method to scan the DOM and fetch all matching instances into a list collection.
- **Counting Elements:** Once a list collection containing targeted `WebElement` components is populated via tag name lookups (e.g., `<a>`, `<img>`, `<button>`, `<input>`), standard Java list utilities like `.size()` can be invoked to instantly retrieve item counts.

Tools Required

- Java JDK
- Eclipse IDE / IntelliJ IDEA
- Selenium WebDriver Server Libraries
- Google Chrome Browser & compatible ChromeDriver

Procedure

Step 1: Create Java Project

1. Open **Eclipse IDE**.
2. Select **File** → **New** → **Java Project**.
3. Enter the project name: `ObjectCounting` and click **Finish**.

Step 2: Add Selenium WebDriver Libraries

Add the required Selenium WebDriver external `.jar` binary component dependencies to your Java project's Build Path under the Libraries tab.

Step 3: Create Package and Class

1. Right-click the `src` source folder directory tree pane and create a new package named `objectcounting`.

2. Right-click the newly generated package folder, add a new Java class named `ObjectCounting`, select the checkbox for `public static void main(String[] args)`, and click **Finish**.

Step 4: Write and Run the Code

Implement the element tracking logic loop inside your class file, save structural changes using `Ctrl + S`, and execute as a standard Java Application.

Code Implementation

```
package objectcounting;

import java.util.List;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;

public class ObjectCounting {
    public static void main(String[] args) {
        // Launch Chrome browser instance
        WebDriver driver = new ChromeDriver();

        // Maximize window container boundaries
        driver.manage().window().maximize();

        // Open the targeted practice web page URL location
        driver.get("https://selenium.dev");

        // Identify all active hyperlink links present in DOM layout via tag locator
        List<WebElement> links = driver.findElements(By.tagName("a"));

        // Identify all graphical image tags
        List<WebElement> images = driver.findElements(By.tagName("img"));

        // Identify all button interaction elements
        List<WebElement> buttons = driver.findElements(By.tagName("button"));

        // Identify all fundamental text box/checkbox input field structures
        List<WebElement> inputs = driver.findElements(By.tagName("input"));

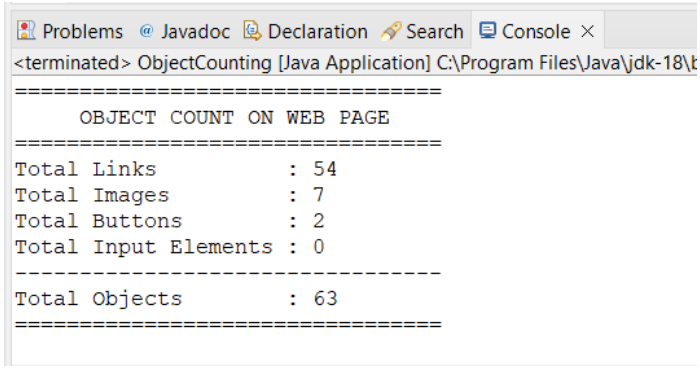
        // Display formatted categorized inventory count logs to console stream
        System.out.println("================================");
        System.out.println("     OBJECT COUNT ON WEB PAGE    ");
        System.out.println("================================");
        System.out.println("Total Links : " + links.size());
        System.out.println("Total Images : " + images.size());
        System.out.println("Total Buttons : " + buttons.size());
        System.out.println("Total Input Elements : " + inputs.size());

        // Calculate the aggregate total number of tracked objects
        int totalObjects = links.size()
                         + images.size()
                         + buttons.size()
                         + inputs.size();

        System.out.println("--------------------------------");
        System.out.println("Total Objects : " + totalObjects);
        System.out.println("================================");

        // Close the browser session and kill background driver engine processes cleanly
        driver.quit();
    }
}
```

Console Execution Output



Expected Result

The automated test script successfully navigates to the target site, crawls the underlying DOM document layout structure using general tag name locators, calculates element counts, and outputs the structured matrix details to the console log interface.

9 Practical No. 9: List and Combo Box Item Verification

Aim

To automate the process of identifying a list or dropdown box on a web page using Selenium WebDriver, determine the total number of items, verify the default selected value, and perform selection testing.

Theory

- **Dropdown / Combo Box:** A graphical user interface element that allows users to select one or more options from a predefined list. In HTML, dropdowns are typically created using `<select>` parent containers wrapped around inner `<option>` element nodes.
- **Selenium Select Class:** Selenium provides a dedicated `Select` class within the `org.openqa.selenium.support.ui` package to interact natively with standard dropdown elements. It includes built-in methods to easily count available options, fetch current selections, or switch choices by visible text, index, or value attributes.

Tools Required

- Java JDK
- Eclipse IDE / IntelliJ IDEA
- Selenium WebDriver Server Libraries
- Google Chrome Browser & compatible ChromeDriver

Procedure

Step 1: Create Java Project

1. Open **Eclipse IDE**.
2. Select **File** → **New** → **Java Project**.
3. Name the project `ListComboBoxVerification` and click **Finish**.

Step 2: Add Selenium WebDriver Libraries

Right-click your newly generated project folder → select **Build Path** → **Configure Build Path**. Under the **Libraries** tab, select **Classpath**, click **Add External JARs...**, import your core Selenium external `.jar` binary files, and click **Apply and Close**.

Step 3: Create Package and Class Structure

1. Right-click the `src` folder pane and create a new package named `listcombobox`.
2. Right-click the new package, add a new Java class named `ListComboBoxVerification`, select the checkbox for `public static void main(String[] args)`, and click **Finish**.

Step 4: Write and Run the Code

Implement the dropdown selection verification logic loop inside your class file, save structural changes using `Ctrl + S`, and execute as a standard Java Application.

Code Implementation

```java
package listcombobox;

import java.util.List;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.Select;

public class ListComboBoxVerification {
    public static void main(String[] args) throws InterruptedException {
        // Launch Chrome browser instance
        WebDriver driver = new ChromeDriver();

        // Maximize browser container layout boundaries
        driver.manage().window().maximize();

        // Open the targeted practice web page URL location
        driver.get("https://selenium.dev");

        // Locate the target dropdown element structure using name locator
        WebElement dropdown = driver.findElement(By.name("my-select"));

        // Instantiate the Select helper wrapper class around the targeted element
        Select select = new Select(dropdown);
```

```
        // Extract all option choices populated inside the select block
        List<WebElement> options = select.getOptions();

        // Print header summary data logs to console stream
        System.out.println("================================");
        System.out.println("  LIST / COMBO BOX VERIFICATION  ");
        System.out.println("================================");
        System.out.println("Total Items : " + options.size());

        // Loop through and display the visible text value of each item node
        System.out.println("\nDropdown Items:");
        for (WebElement option : options) {
            System.out.println(option.getText());
        }

        // Identify and log the application's native default selected value
        WebElement defaultOption = select.getFirstSelectedOption();
        System.out.println("\nDefault Selected Value : " + defaultOption.getText());

        // Update selection choice by visible text string metric
        select.selectByVisibleText("Two");

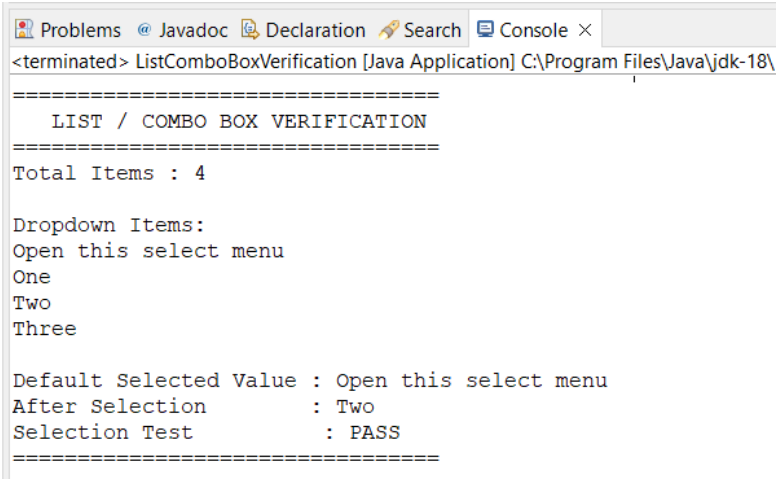
        // Fetch and print the updated selected option node state details
        WebElement selectedOption = select.getFirstSelectedOption();
        System.out.println("After Selection        : " + selectedOption.getText());

        // Validation check step condition
        if (selectedOption.getText().equals("Two")) {
            System.out.println("Selection Test         : PASS");
        } else {
            System.out.println("Selection Test         : FAIL");
        }
        System.out.println("================================");

        // Hold browser open for a brief observation interval period
        Thread.sleep(5000);

        // Terminate active engine execution threads cleanly
        driver.quit();
    }
}
```

Console Execution Output



Expected Result

The automated Java application successfully identifies the list combo box element layout, parses all structural option items, accurately reads the initial default placeholder text string, updates the selection choice value cleanly, and outputs a passing verification test log status.

10 Practical No. 10: Checkbox Identification and Validation

Aim

To identify and count the total number of checkboxes on a web page, count checked and unchecked checkboxes, perform dynamic checkbox selection, and validate the selected checkbox using Selenium WebDriver.

Theory

- **Checkbox Element:** A graphical user interface component that allows users to select or deselect binary choices. In HTML, these elements are structured using the `<input type="checkbox">` tag layout pattern.
- **Selenium Validation Methods:**

- `findElements(By.xpath("//input[@type='checkbox']"))` : Locates all matching checkbox structures across the page DOM tree and stores them in a list collection.
- `isSelected()` : A boolean method that queries whether a given checkbox or radio button component is currently selected (`true`) or unselected (`false`).
- `click()` : Toggles the state of a checkbox by clicking it.

- **Dynamic Selection:** The practice of systematically evaluating and selecting an unchecked element at runtime based on its state, rather than hardcoding a fixed target index or element.

Tools Required

- Java JDK (Version 8 or above)
- Eclipse IDE / IntelliJ IDEA
- Selenium WebDriver Server & Java Client Libraries
- Google Chrome Browser & compatible ChromeDriver
- Internet Connection

Target Website Details

- **URL:** `https://lambdatest.com`

Procedure

1. Open **Eclipse IDE**.
2. Navigate to **File** → **New** → **Java Project** and name it `CheckboxTesting` .
3. Create a clean package path layout named `checkbox` .
4. Create a Java Class named `CheckboxIdentification` within your package folder.
5. Add your core Selenium external `.jar` binary component dependencies to your Java project's Build Path under the Libraries tab.
6. Write the automation program to fetch all checkboxes, loop through them to verify initial selection patterns, check unchecked elements, and log state differences.

Code Implementation

```
package checkbox;

import java.time.Duration;
import java.util.List;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;

public class CheckboxIdentification {
    public static void main(String[] args) throws InterruptedException {
        // Launch Chrome browser instance
        WebDriver driver = new ChromeDriver();

        // Maximize browser container layout boundaries
        driver.manage().window().maximize();

        // Configure Implicit synchronization timeout interval
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));

        // Open the targeted playground checkbox demo web page
        driver.get("https://lambdatest.com");

        // Locate all checkboxes on the page using a type attribute locator path
        List<WebElement> checkboxes = driver.findElements(By.xpath("//input[@type='checkbox']"));

        // Extract aggregate checkbox collection inventory sizes
        int totalCheckboxes = checkboxes.size();
        System.out.println("==================================");
        System.out.println("     CHECKBOX IDENTIFICATION      ");
        System.out.println("==================================");
        System.out.println("Total Number of Checkboxes: " + totalCheckboxes);

        // Loop through and click the first 5 unchecked checkboxes found
        for (int i = 0; i < 5 && i < checkboxes.size(); i++) {
            if (!checkboxes.get(i).isSelected()) {
                checkboxes.get(i).click();
            }
        }

        // Count checked vs unchecked states within the element list matrix
        int checkedCount = 0;
        int uncheckedCount = 0;
        for (WebElement checkbox : checkboxes) {
            if (checkbox.isSelected()) {
                checkedCount++;
            } else {
                uncheckedCount++;
```

```
            }
        }

        // Display initial selection count states
        System.out.println("Checked Checkboxes: " + checkedCount);
        System.out.println("Unchecked Checkboxes: " + uncheckedCount);

        // Perform a dynamic runtime selection test against an unchecked target
        for (WebElement checkbox : checkboxes) {
            if (!checkbox.isSelected()) {
                checkbox.click();
                System.out.println();
                System.out.println("An unchecked checkbox was selected.");

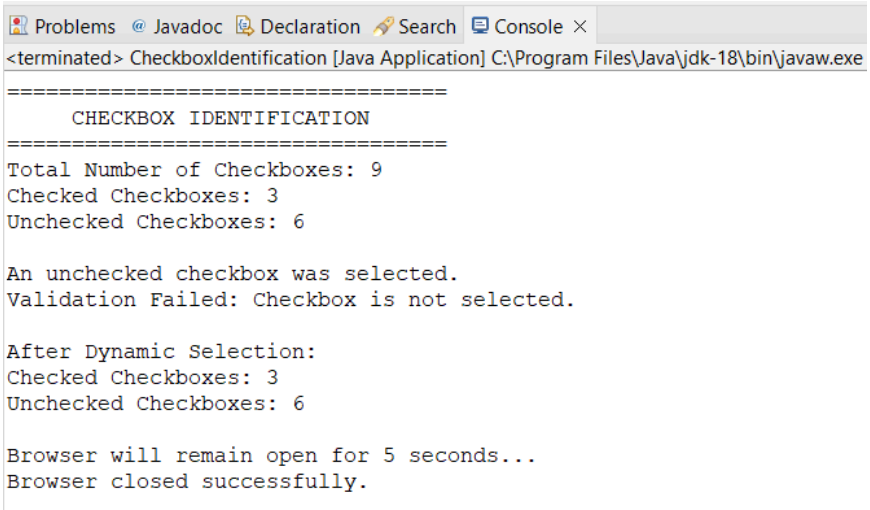
                // Assert validation status state change parameters immediately
                if (checkbox.isSelected()) {
                    System.out.println("Validation Successful: Checkbox is selected.");
                } else {
                    System.out.println("Validation Failed: Checkbox is not selected.");
                }
                break; // Break the cycle loops cleanly after finding one item
            }
        }

        // Recalculate totals across the matrix to verify adjustments worked
        checkedCount = 0;
        uncheckedCount = 0;
        for (WebElement checkbox : checkboxes) {
            if (checkbox.isSelected()) {
                checkedCount++;
            } else {
                uncheckedCount++;
            }
        }
        System.out.println();
        System.out.println("After Dynamic Selection:");
        System.out.println("Checked Checkboxes: " + checkedCount);
        System.out.println("Unchecked Checkboxes: " + uncheckedCount);

        // Keep browser open for execution review observation
        System.out.println();
        System.out.println("Browser will remain open for 5 seconds...");
        Thread.sleep(5000);

        // Terminate application driver engine workspace instance processes
        driver.quit();
        System.out.println("Browser closed successfully.");
    }
}
```

Console Execution Output



Expected Result

The automated Java program successfully identifies all checkbox locator input tags inside the target playground DOM, loops through checking unselected boxes dynamically, updates aggregate checkbox array validation count statistics accurately, and completes execution without script errors.

11 Practical no. 11 Dynamic Web Table Handling and Sorting Validation

Aim:

To automate extraction of data from a dynamic web table using Selenium WebDriver, validate sorting functionality, and verify filtered results.

Software/Tools Required

- Java
- Selenium WebDriver
- Eclipse/IntelliJ IDEA
- Google Chrome
- ChromeDriver

Procedure

1. Create a Java project and add Selenium WebDriver dependencies.
2. Create a package named `webtable`.
3. Create a class named `DynamicWebTable`.
4. Launch the Chrome browser using Selenium WebDriver.

Program:

```
package webtable;

import java.time.Duration;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;

public class DynamicWebTable {
    public static void main(String[] args) throws InterruptedException {
        // Launch Chrome
        WebDriver driver = new ChromeDriver();

        // Maximize browser
        driver.manage().window().maximize();

        // Implicit wait
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));

        // Open web table webpage
        driver.get("https://lambdatest.com");

        System.out.println("=======================================");
        System.out.println("     DYNAMIC WEB TABLE HANDLING       ");
        System.out.println("=======================================");

        // Find total number of rows
        int totalRows = driver.findElements(By.xpath("//table[@id='example']/tbody/tr")).size();
        System.out.println("Total Number of Rows: " + totalRows);
        System.out.println();
        System.out.println("Table Data:");

        // Extract table data directly
        List<WebElement> cells = driver.findElements(By.xpath("//table[@id='example']/tbody/tr/td"));
        for (int i = 0; i < cells.size(); i++) {
            System.out.print(cells.get(i).getText() + " | ");

            // Move to next line after every 4 columns
            if ((i + 1) % 4 == 0) {
                System.out.println();
            }
        }

        // Get names before sorting
        List<WebElement> nameCells = driver.findElements(By.xpath("//table[@id='example']/tbody/tr/td[1]"));
        List<String> beforeSorting = new ArrayList<>();
        for (WebElement name : nameCells) {
            beforeSorting.add(name.getText());
        }

        // Click Name column
        WebElement nameHeader = driver.findElement(By.xpath("//table[@id='example']//th[contains(.,'Name')]"));
        nameHeader.click();
        Thread.sleep(2000);

        // Get names after sorting
        List<WebElement> sortedNameCells = driver.findElements(By.xpath("//table[@id='example']/tbody/tr/td[1]"));
        List<String> afterSorting = new ArrayList<>();
        for (WebElement name : sortedNameCells) {
            afterSorting.add(name.getText());
        }

        // Create expected sorted list
        List<String> expectedNames = new ArrayList<>(afterSorting);
        Collections.sort(expectedNames, String.CASE_INSENSITIVE_ORDER);

        System.out.println();
        System.out.println("Sorting Validation:");
```

```
        if (afterSorting.equals(expectedNames)) {
            System.out.println("Sorting Validation Successful: Data is sorted correctly.");
        } else {
            System.out.println("Sorting Validation Failed: Data is not sorted correctly.");
        }

        // Search / Filter
        WebElement searchBox = driver.findElement(By.xpath("//input[@type='search']"));
        searchBox.clear();
        searchBox.sendKeys("London");
        Thread.sleep(2000);

        // Find filtered rows
        List<WebElement> filteredRows = driver.findElements(By.xpath("//table[@id='example']/tbody/tr"));
        boolean filterValid = true;

        System.out.println();
        System.out.println("Filtered Results:");
        for (int i = 0; i < filteredRows.size(); i++) {
            // Re-find the row every time
            String rowText = driver.findElement(By.xpath("//table[@id='example']/tbody/tr[" + (i + 1) + "]")).getText();
            System.out.println(rowText);
            if (!rowText.toLowerCase().contains("london")) {
                filterValid = false;
            }
        }

        System.out.println();
        System.out.println("Filtering Validation:");
        if (filterValid && filteredRows.size() > 0) {
            System.out.println("Filtering Validation Successful: Correct results displayed.");
        } else {
            System.out.println("Filtering Validation Failed: Incorrect results displayed.");
        }

        // Keep browser open
        System.out.println();
        System.out.println("Browser will remain open for 5 seconds...");
        Thread.sleep(5000);

        // Close browser
        driver.quit();
        System.out.println("Browser closed successfully.");
    }
}
```

Output:

```
---------------------------------------
     DYNAMIC WEB TABLE HANDLING
=======================================
Total Number of Rows: 24

Table Data:
A. Bennett | Photographer | Chicago | 61 |
A. Nelson | Physicist | China | 17 |
A. Satou | Accountant | Tokyo | 33 |
B. Greer | Software Engineer | London | 41 |
B. Williamson | Integration Specialist | New York | 61 |
C. Hurst | Javascript Developer | San Francisco | 39 |
C. Kelly | Senior Javascript Developer | Edinburgh | 22 |
C. Marshall | Regional Director | San Francisco | 36 |
C. Turner | Auditor | New York | 30 |
D. Rios | Personnel Lead | Edinburgh | 35 |

Sorting Validation:
Sorting Validation Failed: Data is not sorted correctly.

Filtered Results:
T. Fitzpatrick Regional Director London 19
M. Silva Marketing Designer London 66
J. Gaines Office Manager London 30
H. Kennedy Senior Marketing Designer London 43
B. Greer Software Engineer London 41

Filtering Validation:
Filtering Validation Successful: Correct results displayed.

Browser will remain open for 5 seconds...
Browser closed successfully.
```

12 Practical No. 12: File Upload and Download Automation

Aim

To automate file upload and download functionality using Selenium WebDriver and Robot Class, validate successful file upload, and verify the existence of the downloaded file.

Tools Required

- Eclipse IDE

- Java JDK
- Selenium WebDriver
- Chrome Browser
- Selenium Server (`selenium-server-4.44.0.jar`)
- Robot Class

Procedure

1. Open **Eclipse IDE**.
2. Create a New Java Project named `File Upload Download`.
3. Configure the Project Build Path:
   - Right-click the project → **Build Path** → **Configure Build Path** → **Libraries** → **Add External JARs**.
   - Select `selenium-server-4.44.0.jar` (or version 4.46.0.jar as configured) and click **Apply and Close**.
4. Create a package named `fileupload`.
5. Create a Java class named `FileUploadDownload` and check the box for `public static void main(String[] args)`.
6. Create a text file named `sample.txt` on the Desktop. Add some sample text and save the file.
7. Write the complete Java program given below into the class file. Change the file path in the program according to your computer's username.
8. Save the program (`Ctrl + S`) and select **Run As** → **Java Application**.

Code Implementation

```
package fileupload;

import java.io.File;
import java.time.Duration;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;

public class FileUploadDownload {
    public static void main(String[] args) throws InterruptedException {
        // ====================================================
        // 1. FILE PATHS
        // ====================================================
        String uploadFilePath = "C:/Users/PVGIT/Desktop/sample.txt";
        String downloadFolder = "C:/Users/PVGIT/Downloads";
        String downloadedFile = downloadFolder + "/sample.txt";

        // ====================================================
        // 2. CHECK WHETHER UPLOAD FILE EXISTS
        // ====================================================
        File uploadFile = new File(uploadFilePath);
        if (!uploadFile.exists()) {
            System.out.println("ERROR: sample.txt was not found.");
            System.out.println("Check the file path:");
            System.out.println(uploadFilePath);
            return;
        }
        System.out.println("Upload file found successfully.");

        // ====================================================
        // 3. LAUNCH CHROME
        // ====================================================
        WebDriver driver = new ChromeDriver();
        driver.manage().window().maximize();
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
        System.out.println("Chrome browser launched.");

        // ====================================================
        // 4. OPEN FILE UPLOAD PAGE
        // ====================================================
        driver.get("https://herokuapp.com");
        System.out.println();
        System.out.println("======================================");
        System.out.println(" FILE UPLOAD AND DOWNLOAD AUTOMATION");
        System.out.println("======================================");
        System.out.println();
        System.out.println("Starting file upload...");

        // ====================================================
        // 5. LOCATE FILE INPUT
        // ====================================================
        WebElement fileInput = driver.findElement(By.id("file-upload"));

        // ====================================================
        // 6. UPLOAD FILE
        // ====================================================
        fileInput.sendKeys(uploadFilePath);
        System.out.println("sample.txt selected successfully.");

        // ====================================================
        // 7. CLICK UPLOAD BUTTON
        // ====================================================
```

```
        driver.findElement(By.id("file-submit")).click();
        Thread.sleep(2000);

        // ======================================================
        // 8. VALIDATE UPLOAD
        // ======================================================
        WebElement uploadMessage = driver.findElement(By.tagName("h3"));
        String message = uploadMessage.getText();
        System.out.println();
        System.out.println("Upload Status: " + message);
        if (message.equalsIgnoreCase("File Uploaded!")) {
            System.out.println("Upload Validation Successful.");
        } else {
            System.out.println("Upload Validation Failed.");
        }

        // ======================================================
        // 9. OPEN DOWNLOAD PAGE
        // ======================================================
        System.out.println();
        System.out.println("Starting file download...");
        driver.get("https://herokuapp.com");
        Thread.sleep(2000);

        // ======================================================
        // 10. FIND sample.txt DOWNLOAD LINK
        // ======================================================
        WebElement downloadFile = driver.findElement(By.xpath("//a[contains(text(),'sample.txt')]"));

        // ======================================================
        // 11. CLICK DOWNLOAD
        // ======================================================
        downloadFile.click();
        System.out.println("Download link clicked.");

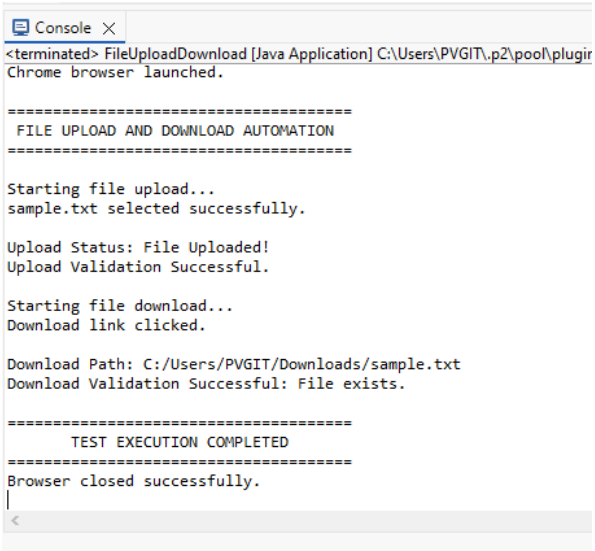
        // Wait for download to complete
        Thread.sleep(5000);

        // ======================================================
        // 12. VERIFY DOWNLOADED FILE
        // ======================================================
        File file = new File(downloadedFile);
        System.out.println();
        System.out.println("Download Path: " + downloadedFile);
        if (file.exists()) {
            System.out.println("Download Validation Successful: File exists.");
        } else {
            System.out.println("Download Validation Failed: File not found.");
        }

        // ======================================================
        // 13. TEST COMPLETION
        // ======================================================
        System.out.println();
        System.out.println("======================================");
        System.out.println("      TEST EXECUTION COMPLETED");
        System.out.println("======================================");

        // Close browser
        driver.quit();
        System.out.println("Browser closed successfully.");
    }
}
```

Console Execution Output



13 Practical No. 13: Screenshot Capture and Logging Mechanism

Aim

To implement screenshot capture on test failure using Selenium WebDriver and integrate Log4j to generate execution logs and analyze error messages.

Tools Required

- Java JDK
- Eclipse IDE
- Selenium WebDriver
- Google Chrome
- ChromeDriver / WebDriverManager
- Log4j 2

Theory

In Selenium automation testing, sometimes a test case fails because an element cannot be found, a page does not load, or some other unexpected error occurs. A screenshot helps us see the exact state of the browser at the time of failure. This makes it easier to understand and debug the problem.

Log4j is a Java logging framework used to record information about program execution. It can record different types of messages such as:

- **INFO** – normal execution information
- **WARN** – warning messages
- **ERROR** – errors or failures
- **DEBUG** – detailed debugging information

In this practical, Selenium is used to perform the test, while Log4j records the execution and error messages. When the test fails, Selenium automatically captures a screenshot and saves it in the screenshots folder.

Procedure

1. Create a Java project named `ScreenshotLoggingProject`.
2. Add Required JAR Files: Right-click the project → **Build Path** → **Configure Build Path** → **Libraries** → **Classpath** → **Add JARs**. Add the required Selenium and Log4j JAR files. For Log4j, make sure you have both `log4j-api-2.x.x.jar` and `log4j-core-2.x.x.jar`.
3. Create the Package: Right-click `src` → **New** → **Package**. Enter `screenshotlogging` and click Finish.
4. Create the Java Class: Right-click the package → **New** → **Class**. Enter `ScreenshotLoggingTest` and click Finish.
5. Create `log4j2.xml`: Right-click the `src` folder → **New** → **File**. Enter `log4j2.xml` and paste the configuration provided below.
6. Write the Java program, save, and run as a Java Application.

Configuration File (`log4j2.xml`)

```
<?xml version="1.0" encoding="UTF-8"?>
<Configuration status="WARN">
    <Appenders>
        <Console name="Console" target="SYSTEM_OUT">
            <PatternLayout pattern="%d{yyyy-MM-dd HH:mm:ss} %-5level %logger{36} - %msg%n"/>
        </Console>
    <File name="FileLogger" fileName="execution.log">
        <PatternLayout pattern="%d{yyyy-MM-dd HH:mm:ss} %-5level %logger{36} - %msg%n"/>
    </File>
    </Appenders>
    <Loggers>
        <Root level="info">
            <AppenderRef ref="Console"/>
            <AppenderRef ref="FileLogger"/>
        </Root>
    </Loggers>
</Configuration>
```

Code Implementation

```
package screenshotlogging;

import java.io.File;
import java.text.SimpleDateFormat;
import java.util.Date;
import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;
import org.openqa.selenium.By;
import org.openqa.selenium.OutputType;
import org.openqa.selenium.TakesScreenshot;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
```

```java
public class ScreenshotLoggingTest {
    private static final Logger logger = LogManager.getLogger(ScreenshotLoggingTest.class);

    public static void main(String[] args) {
        WebDriver driver = null;
        try {
            logger.info("Test execution started.");

            // Start Chrome
            driver = new ChromeDriver();
            logger.info("Chrome browser started.");

            // Open website
            driver.get("https://google.com");
            logger.info("Website opened successfully.");

            // Maximize browser
            driver.manage().window().maximize();
            logger.info("Browser maximized.");

            // Intentionally use an incorrect ID to generate a test failure
            driver.findElement(By.id("wrongElement"));
            logger.info("Element found successfully.");

        } catch (Exception e) {
            logger.error("Test failed.");
            logger.error("Error message: " + e.getMessage());

            // Capture screenshot
            if (driver != null) {
                try {
                    TakesScreenshot screenshot = (TakesScreenshot) driver;
                    File source = screenshot.getScreenshotAs(OutputType.FILE);
                    String timeStamp = new SimpleDateFormat("yyyyMMdd_HHmmss").format(new Date());
                    File destination = new File("screenshots/failure_" + timeStamp + ".png");

                    java.nio.file.Files.copy(
                        source.toPath(),
                        destination.toPath(),
                        java.nio.file.StandardCopyOption.REPLACE_EXISTING
                    );
                    logger.info("Screenshot captured: " + destination.getAbsolutePath());
                } catch (Exception screenshotError) {
                    logger.error("Unable to capture screenshot: " + screenshotError.getMessage());
                }
            }
        } finally {
            if (driver != null) {
                driver.quit();
                logger.info("Browser closed.");
            }
            logger.info("Test execution completed.");
        }
    }
}
```

Console Output

```
Problems  Javadoc  Declaration  Search  Console
<terminated> ScreenshotLoggingTest (1) [Java Application] C:\Program Files\Java\jdk-18\bin\javaw.exe  (1 Sep 2026, 11:22:17 pm – 11:22:22 pm) [pid: 24220]
WARNING: Unable to find version of CDP to use for 152.0.7977.65. You may need to include a dependency on a specific version of the CDP using someth
2026-09-01 23:22:20 INFO  screenshotlogging.ScreenshotLoggingTest - Chrome browser started.
2026-09-01 23:22:21 INFO  screenshotlogging.ScreenshotLoggingTest - Website opened successfully.
2026-09-01 23:22:21 INFO  screenshotlogging.ScreenshotLoggingTest - Browser maximized.
2026-09-01 23:22:21 ERROR screenshotlogging.ScreenshotLoggingTest - Test failed.
2026-09-01 23:22:21 ERROR screenshotlogging.ScreenshotLoggingTest - Error message: no such element: Unable to locate element: {"method":"css select
  (Session info: chrome=152.0.7977.65)
For documentation on this error, please visit: https://www.selenium.dev/documentation/webdriver/troubleshooting/errors#nosuchelementexception
Build info: version: '4.44.0', revision: 'da2039bd14*'
System info: os.name: 'Windows 11', os.arch: 'amd64', os.version: '10.0', java.version: '18'
Driver info: org.openqa.selenium.chrome.ChromeDriver
Command: [cac8227d28d6d7deb36b2f4d47f2c2ac, findElement {using=id, value=wrongElement}]
Capabilities {acceptInsecureCerts: false, browserName: chrome, browserVersion: 152.0.7977.65, chrome: {chromedriverVersion: 152.0.7977.64 (506c834e
Session ID: cac8227d28d6d7deb36b2f4d47f2c2ac
2026-09-01 23:22:21 INFO  screenshotlogging.ScreenshotLoggingTest - Screenshot captured: C:\Users\Kiran\eclipse-workspace\ScreenshotLoggingProject\
2026-09-01 23:22:22 INFO  screenshotlogging.ScreenshotLoggingTest - Browser closed.
2026-09-01 23:22:22 INFO  screenshotlogging.ScreenshotLoggingTest - Test execution completed.
```

14 Practical No. 14: Automation testing framework using Page Object Model (POM)

Aim

To design and implement an automation testing framework using Page Object Model (POM) with Selenium WebDriver and TestNG, by creating reusable page classes and executing modular test cases.

Tools Required

- Java JDK
- Eclipse IDE

- Maven
- Selenium WebDriver
- TestNG
- Google Chrome
- ChromeDriver

Theory

Page Object Model (POM) is a design pattern used in Selenium automation testing. In POM, each web page is represented by a separate Java class. The page class contains the web elements and methods/actions performed on that page, while the test class contains the test cases and assertions.

This makes the automation framework:

- Easy to understand
- Reusable
- Easy to maintain
- Modular
- Less repetitive

For example, instead of writing Google-related code directly inside the test class, we can create a `GooglePage` class and put the page-related actions there.

Procedure

1. **Create Maven Project:** Create a Maven Project in Eclipse. Use Group Id: `com.testing` and Artifact Id: `POMTesting`. The project will automatically contain a `pom.xml` file.
2. **Configure `pom.xml`:** Add Selenium and TestNG dependencies to the `pom.xml`.
3. **Create Page Class:** Inside `src/test/java` → `com.testing` (package), create a Java class named `GooglePage.java`.
4. **Create Test Class:** Now create `SeleniumTest.java` in the same package.
5. **Create `testng.xml`:** Create `testng.xml` in the project root directory to manage execution.

File Implementations

1. `pom.xml`

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://apache.org"
 xmlns:xsi="http://w3.org"
 xsi:schemaLocation="http://apache.org
 https://apache.org">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.testing</groupId>
    <artifactId>POMTesting</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <properties>
        <maven.compiler.source>11</maven.compiler.source>
        <maven.compiler.target>11</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>
    <dependencies>
        <!-- Selenium -->
        <dependency>
            <groupId>org.seleniumhq.selenium</groupId>
            <artifactId>selenium-java</artifactId>
            <version>4.44.0</version>
        </dependency>
        <!-- TestNG -->
        <dependency>
            <groupId>org.testng</groupId>
            <artifactId>testng</artifactId>
            <version>7.11.0</version>
            <scope>test</scope>
        </dependency>
    </dependencies>
    <build>
        <plugins>
            <!-- Maven Compiler Plugin -->
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.13.0</version>
                <configuration>
                    <source>11</source>
                    <target>11</target>
                </configuration>
            </plugin>
            <!-- Maven Surefire Plugin -->
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-surefire-plugin</artifactId>
                <version>3.5.3</version>
```

```
                <configuration>
                    <suiteXmlFiles>
                        <suiteXmlFile>testng.xml</suiteXmlFile>
                    </suiteXmlFiles>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>
```

2. Page Class (`GooglePage.java`)

```
package com.testing;

import org.openqa.selenium.WebDriver;

public class GooglePage {
    WebDriver driver;

    // Constructor
    public GooglePage(WebDriver driver) {
        this.driver = driver;
    }

    // Open Google
    public void openGoogle() {
        driver.get("https://google.com");
    }

    // Get page title
    public String getPageTitle() {
        return driver.getTitle();
    }
}
```

3. Test Class (`SeleniumTest.java`)

```
package com.testing;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.Assert;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;

public class SeleniumTest {
    WebDriver driver;
    GooglePage googlePage;

    @BeforeMethod
    public void setUp() {
        driver = new ChromeDriver();
        driver.manage().window().maximize();
        googlePage = new GooglePage(driver);
        System.out.println("Browser opened successfully.");
    }

    @Test
    public void verifyGoogleTitle() {
        googlePage.openGoogle();
        String actualTitle = googlePage.getPageTitle();
        System.out.println("Page Title: " + actualTitle);
        Assert.assertTrue(
            actualTitle.contains("Google"),
            "Google title was not found."
        );
        System.out.println("Test Passed.");
    }

    @AfterMethod
    public void tearDown() {
        if (driver != null) {
            driver.quit();
        }
        System.out.println("Browser closed.");
    }
}
```

4. Execution Suite (`testng.xml`)

```
<?xml version="1.0" encoding="UTF-8"?>
<suite name="Selenium Test Suite">
    <test name="Google Test">
        <classes>
            <class name="com.testing.SeleniumTest"/>
        </classes>
    </test>
</suite>
```

Output

```
Problems  Javadoc  Declaration  Search  Console ×
<terminated> C:\Program Files\Java\jdk-18\bin\javaw.exe (7 Sep 2026, 8:42:16 pm) [pid: 22856]
Browser opened successfully.
Page Title: Google
Test Passed.
Browser closed.
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 10.11 s -- in TestSuite
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  14.271 s
[INFO] Finished at: 2026-09-07T20:42:33+05:30
[INFO] ------------------------------------------------------------------------
```

15 Practical No. 15: TestNG Framework Implementation and Reporting

Aim

To create and execute automated test cases using TestNG annotations, group test cases, define test execution priorities, and generate an HTML test report using Selenium WebDriver and TestNG.

Tools Required

- Java JDK 11 or above
- Eclipse IDE
- Maven
- Selenium WebDriver
- TestNG
- WebDriverManager
- Google Chrome
- Internet connection

Theory

TestNG is a testing framework for Java that is used to create, organize, execute, and report automated test cases. TestNG provides annotations that control the execution of test methods. Some commonly used annotations are:

- `@BeforeMethod` – executes before each test method.
- `@Test` – identifies a method as a test case.
- `@AfterMethod` – executes after each test method.

TestNG also supports priorities and groups. Priority is used to define the order of test execution, while groups are used to categorize related test cases. TestNG can also generate an HTML report after test execution. The report shows the number of tests executed, passed, failed, and skipped.

Procedure

1. **Create a Maven Project:** Open Eclipse IDE → **File** → **New** → **Maven Project**. Select "Create a simple project" and enter Group Id: `com.testing` and Artifact Id: `TestNGFramework`.
2. **Create the Package:** Inside `src/test/java`, right-click and create a package named `com.testing`.
3. **Create the Java Class:** Inside the package, create a class named `TestNGFrameworkTest`.
4. **Configure `pom.xml`:** Open the project's `pom.xml` file and add dependencies for Selenium, TestNG, and WebDriverManager.
5. **Update Maven Project:** Right-click the project → **Maven** → **Update Project** to download dependencies.
6. **Write the Java Program:** Paste the complete TestNG code into your class file.
7. **Create `testng.xml`:** Create the XML execution suite in the project root folder.

File Implementations

1. `pom.xml`

```
<?xml version="1.0" encoding="UTF-8"?>
<project
    xmlns="http://apache.org"
```

```
    xmlns:xsi="http://w3.org"
 xsi:schemaLocation="http://apache.org
 https://apache.org">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.testing</groupId>
    <artifactId>TestNGFramework</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <properties>
        <maven.compiler.source>11</maven.compiler.source>
        <maven.compiler.target>11</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>
    <dependencies>
        <!-- Selenium -->
        <dependency>
            <groupId>org.seleniumhq.selenium</groupId>
            <artifactId>selenium-java</artifactId>
            <version>4.44.0</version>
        </dependency>
        <!-- TestNG -->
        <dependency>
            <groupId>org.testng</groupId>
            <artifactId>testng</artifactId>
            <version>7.11.0</version>
            <scope>test</scope>
        </dependency>
        <!-- WebDriverManager -->
        <dependency>
            <groupId>io.github.bonigarcia</groupId>
            <artifactId>webdrivermanager</artifactId>
            <version>6.3.3</version>
        </dependency>
    </dependencies>
</project>
```

2. Test Class (`TestNGFrameworkTest.java`)

```
package com.testing;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.Assert;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;
import io.github.bonigarcia.wdm.WebDriverManager;

public class TestNGFrameworkTest {
    WebDriver driver;

    // Runs before every test case
    @BeforeMethod
    public void setUp() {
        WebDriverManager.chromedriver().setup();
        driver = new ChromeDriver();
        driver.manage().window().maximize();
        driver.get("https://google.com");
        System.out.println("Browser opened successfully");
    }

    // Test Case 1
    @Test(priority = 1, groups = {"smoke"})
    public void verifyTitle() {
        String title = driver.getTitle();
        System.out.println("Page Title: " + title);
        Assert.assertTrue(title.contains("Google"));
        System.out.println("Title test passed");
    }

    // Test Case 2
    @Test(priority = 2, groups = {"smoke"})
    public void verifyURL() {
        String url = driver.getCurrentUrl();
        System.out.println("Current URL: " + url);
        Assert.assertTrue(url.contains("google.com"));
        System.out.println("URL test passed");
    }

    // Test Case 3
    @Test(priority = 3, groups = {"regression"})
    public void verifyPage() {
        String title = driver.getTitle();
        System.out.println("Checking page title...");
        Assert.assertNotNull(title);
        System.out.println("Page verification test passed");
    }

    // Runs after every test case
    @AfterMethod
    public void tearDown() {
        if (driver != null) {
            driver.quit();
            System.out.println("Browser closed");
        }
```

```
    }
}
```

3. Execution Suite (`testng.xml`)

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "https://testng.org">
<suite name="TestNG Suite">
    <test name="Automation Tests">
        <groups>
            <run>
                <include name="smoke"/>
                <include name="regression"/>
            </run>
        </groups>
        <classes>
            <class name="com.testing.TestNGFrameworkTest"/>
        </classes>
    </test>
</suite>
```

Output

```
<terminated> C:\Program Files\Java\jdk-18\bin\javaw.exe (8 Sep 2026, 12:36:57 am) [pid: 9092]
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running com.testing.TestNGFrameworkTest
SLF4J(W): No SLF4J providers were found.
SLF4J(W): Defaulting to no-operation (NOP) logger implementation
SLF4J(W): See https://www.slf4j.org/codes.html#noProviders for further details.
Sep 08, 2026 12:37:34 AM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find an exact match for CDP version 152, returning the closest version; found: 148; Please update to
Browser opened successfully
Page Title: Google
Title test passed
Browser closed
Sep 08, 2026 12:37:37 AM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find an exact match for CDP version 152, returning the closest version; found: 148; Please update to
Browser opened successfully
Current URL: https://www.google.com/
URL test passed
Browser closed
Sep 08, 2026 12:37:39 AM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find an exact match for CDP version 152, returning the closest version; found: 148; Please update to
Browser opened successfully
Checking page title...
Page verification test passed
Browser closed
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 34.01 s -- in com.testing.TestNGFrameworkTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  42.450 s
[INFO] Finished at: 2026-09-08T00:37:41+05:30
[INFO] ------------------------------------------------------------------------
```